Universidad de San Carlos de Guatemala
Escuela de Ciencias y Sistemas
Introduccion a la Programacion y Computacion 1 Seccion E

Manual Tecnico: Quetzal Space Defender

Practica 2

Estudiante: Juan Esteban Ruiz Villatoro
Carnet: 202400087

12 de octubre de 2026

Índice

1. Introduccion **2**

2. Estructura del Proyecto y Paquetes **2**

3. Librerias Implementadas **2**

4. Logica de Hilos y Concurrencia **3**

5. Manejo de Persistencia Local **3**

1. Introduccion

El presente documento detalla la arquitectura, estructura de clases, logica de hilos, gestion de archivos locales y diseno tecnico implementado en el desarrollo de la aplicacion **Quetzal Space Defender**. El proyecto ha sido desarrollado en lenguaje Java utilizando la libreria grafica Swing, cumpliendo estrictamente con los requerimientos de persistencia local y concurrencia academica.

2. Estructura del Proyecto y Paquetes

El codigo fuente se encuentra organizado bajo el paradigma Modelo-Vista-Controlador (MVC), estructurado en los siguientes paquetes principales dentro de la carpeta `src/`:

- `modelo`: Contiene las clases de entidades del dominio y enumeraciones clave:
  - `Piloto.java`: Define los atributos del jugador (nombre, puntaje, tiempo, nivel).
  - `TipoNave.java`: Encapsula los tipos de naves, dificultades y cadencias de disparo.
  - `Disparo.java`: Representa los proyectiles emitidos durante el juego.
  - `Obstaculo.java`: Modela los elementos interactivos y amenazas en el campo de batalla.
- `controlador`: Contiene la logica de persistencia y manejo de archivos.
  - `GestorArchivos.java`: Encargado de la lectura y escritura sincrona en `puntajes.txt` usando `java.io`.
- `servicio`: Contiene la logica de negocio y reglas de la aplicacion.
  - `PilotoService.java`: Administra el registro, validacion y almacenamiento de pilotos.
- `vista`: Contiene las interfaces graficas desarrolladas con Java Swing.
  - `MenuPrincipal.java`: Pantalla de inicio con opciones de navegacion.
  - `SeleccionNave.java`: Interfaz para asociar piloto y elegir el modelo de nave/dificultad.
  - `CampoBatalla.java`: Ventana principal de juego, bucle de hilos y renderizado grafico.
  - `ReportesVentana.java`: Muestra el top de puntajes ordenados.

3. Librerias Implementadas

El sistema se construyo haciendo uso exclusivamente de las librerias nativas del kit de desarrollo de Java (JDK), evitando frameworks externos o librerias de terceros:

- `javax.swing.*` y `java.awt.*`: Para la construccion de interfaces graficas de usuario (ventanas, paneles, botones, etiquetas y renderizado en 2D).
- `java.io.*`: Para la persistencia de datos local mediante archivos planos (`puntajes.txt`).
- `java.lang.Thread`: Para el manejo de concurrencia nativa en el movimiento de objetos y cadencias de disparo.

4. Logica de Hilos y Concurrencia

El juego implementa hilos nativos para gestionar la fluidez del movimiento y los tiempos de disparo segun la configuracion de la nave seleccionada en el enum `TipoNave.java`:

```
public enum TipoNave {
    EXPLORADOR("Explorador", "Facil", 2000, 20),
    CAZA_ESTELAR("Caza Estelar", "Normal", 1000, 15),
    ACORAZADO("Acorazado", "Dificil", 300, 10);

    private final String nombre;
    private final String dificultad;
    private final int tiempoDisparoMs;
    private final int velocidadMovimiento;

    TipoNave(String nombre, String dificultad, int tiempoDisparoMs,
   int velocidadMovimiento) {
        this.nombre = nombre;
        this.dificultad = dificultad;
        this.tiempoDisparoMs = tiempoDisparoMs;
        this.velocidadMovimiento = velocidadMovimiento;
    }

    public int getTiempoDisparoMs() { return tiempoDisparoMs; }
    public int getVelocidadMovimiento() { return velocidadMovimiento
   ; }
}
```

Listing 1: Configuracion de Hilos en TipoNave.java

5. Manejo de Persistencia Local

La persistencia de los marcadores se realiza de forma local mediante `GestorArchivos.java`, el cual lee y escribe sobre el archivo plano `puntajes.txt` implementando algoritmos de ordenamiento manual (Bubble Sort) para generar los reportes de mejores puntajes sin necesidad de bases de datos externas.